

EEG-Based Emotion Recognition — DEAP Reproduction

Reproduction of the methodology described in:

Mouazen, B.; Benali, A.; Chebchoub, N.T.; Abdelwahed, E.H.; De Marco, G. "Enhancing EEG-Based Emotion Detection with Hybrid Models: Insights from DEAP Dataset Applications." *Sensors* 2025, 25(6), 1827. DOI: [10.3390/s25061827](https://doi.org/10.3390/s25061827)

This is the paper underlying the code report published in [Chabachib/EEG-Based-Emotion-Detection](#), run against the DEAP dataset (mirrored on Kaggle as [manh123df/deap-dataset](#)).

1. What this reproduces

Aspect	Paper	This repo
Dataset	DEAP: 32 subjects, 40 trials, EEG + peripheral	<code>dataset.py</code> loads the preprocessed <code>s01.dat...</code> <code>s32.dat</code>
Labels	valence / arousal / dominance / liking (1–9)	<code>binarize_labels()</code> → binary valence, binary arousal, 4-class quadrant
Preprocessing	band-pass filter → ICA (EOG/EMG removal) → normalization → windowing	<code>preprocessing/preprocessing.py</code> , steps 1–5
Features	STFT + Wavelet Transform ("frequency domain ... to capture temporal dynamics")	<code>features/feature_extraction.py</code> : PSD, Differential Entropy, Hjorth, DWT, STFT
Models	KNN, SVM, DT, RF, BiLSTM, GRU, CNN, autoencoders, transformers	<code>models/models.py</code>
Headline hybrid comparison model	AlexNet + DenseNet feature fusion → PCA → multi-class SVM (95.54% valence / 97.26% arousal)	<code>models/hybrid_alexnet_densenet.py</code>
Interpretability	SHAP	<code>evaluation/evaluate.py</code> : <code>plot_shap_summary</code>
Reported accuracy range	85–95% (up to 90% for the KNN/SVM/DT/RF/BiLSTM/GRU/CNN/AE/Transformer sweep)	Compare with <code>evaluation/evaluate.py</code> : <code>compare_with_paper</code>

Explicit assumptions (paper under-specifies these — flagged, not hidden)

The paper describes *what* techniques were used but not every hyperparameter. Where unspecified, we used the most common choice in the wider DEAP emotion-recognition literature (matching what the linked GitHub repos in this space also use):

1. **Band-pass range:** 4–45 Hz (matches the DEAP-preprocessed release and the theta/alpha/beta/gamma bands used downstream).
2. **ICA artifact rejection rule:** components whose correlation with the Fp1/Fp2 frontal channels exceeds 0.6 are zeroed (standard EOG-proxy heuristic when no dedicated EOG channel is passed to ICA). Note the DEAP-preprocessed `.dat` files already had EOG removed by the DEAP authors, so this step is close to a no-op there — it's included for completeness and for use with raw `.bdf` recordings.
3. **Window length / overlap:** 4 s windows, 50% overlap.
4. **Label thresholding:** valence/arousal binarized at the scale midpoint (>5 = "high").
5. **Cross-validation:** 10-fold **subject-dependent** CV as the primary protocol (all subjects' windows pooled, stratified by class), with an optional **Leave-One-Subject-Out (LOSO)** mode for subject-independent evaluation (`--cv loso`). The paper does not state which of these it used — both are provided so the difference (subject-dependent scores are typically much higher) can be examined directly.
6. **Classical ML hyperparameters:** RBF-SVM ($C=10$), Random Forest (300 trees), Decision Tree (max depth 12), KNN ($k=5$, distance-weighted) — reasonable literature defaults, meant to be tuned.
7. **Deep model architectures:** 2-layer BiLSTM/GRU (hidden=128), a 3-block 1D-CNN, a 2-layer Transformer encoder, and a convolutional autoencoder — these are not given layer-by-layer in the paper, so sizes were chosen to be broadly comparable in capacity across models for a fair comparison.
8. **Hybrid AlexNet+DenseNet+PCA+SVM image construction:** EEG windows are converted into images via per-channel-averaged STFT log-magnitude spectrograms, resized to 224×224 and replicated to 3 "RGB" channels, since the cited hybrid model is architecturally a 2D image-CNN pipeline and the paper does not specify the exact EEG $\rightarrow$ image transform.

2. Project structure

```
project/
├── data/                # place DEAP s01.dat ... s32.dat here (not committed)
├── notebooks/           # exploratory / quickstart notebook
├── preprocessing/
│   └── preprocessing.py # band-pass filter, ICA, baseline removal,
normalization, windowing
├── features/
│   └── feature_extraction.py # PSD, Differential Entropy, Hjorth, Wavelet
(DWT), STFT
```

```

├── models/
│   ├── models.py                # KNN, SVM, DT, RF, BiLSTM, GRU, CNN,
│   │                               Autoencoder, Transformer
│   └── hybrid_alexnet_densenet.py # AlexNet+DenseNet -> PCA -> multi-class
├── SVM
├── training/
│   └── train.py                 # CLI: preprocess -> features -> CV -> train -> save
├── results
├── evaluation/
│   └── evaluate.py             # confusion matrix, ROC, training curves, t-SNE,
├── feature importance, SHAP
├── figures/                    # output plots land here
├── results/                    # output metrics (JSON) land here
├── dataset.py                  # DEAP I/O + label binarization
├── utils.py                    # seeding, device, DEAP metadata
├── requirements.txt
└── README.md

```

3. Installation

```

python -m venv env
source env/bin/activate          # Windows: env\Scripts\activate
pip install -r requirements.txt

```

GPU is optional but strongly recommended for the deep models and the hybrid AlexNet/DenseNet pipeline (ImageNet-pretrained backbones).

4. Dataset preparation

1. Download the **preprocessed** DEAP release, `data_preprocessed_python.zip` (s01.dat ... s32.dat), either from:
 - the official DEAP site (registration required), or
 - the Kaggle mirror: <https://www.kaggle.com/datasets/manh123df/deap-dataset>
2. Unzip so that the files sit directly under `data/`:

```

data/s01.dat
data/s02.dat
...
data/s32.dat

```

3. No manual conversion is needed — `dataset.load_preprocessed_subject` reads the pickle format directly (`encoding="latin1"`, since the files were pickled under Python 2).

If instead you only have the **raw** .bdf recordings, use `dataset.load_raw_subject(path)` (requires mne), which reads the file directly, applies the BioSemi 10-20 montage, and can be dropped into the same `run_pipeline` from there.

5. Training

```
# Classical ML, subject-dependent 10-fold CV, binary valence classification
python training/train.py --data-dir data/ --model random_forest --task valence
--cv subject_dependent

# Classical ML, arousal, Leave-One-Subject-Out
python training/train.py --data-dir data/ --model svm --task arousal --cv loso

# Deep learning
python training/train.py --data-dir data/ --model bilstm --task valence --epochs
30 --batch-size 64
python training/train.py --data-dir data/ --model cnn --task quadrant --epochs
30

# Quick smoke test on a handful of subjects before committing to a full run
python training/train.py --data-dir data/ --model knn --task valence --subjects
3
```

Available `--model` values: `knn`, `svm`, `decision_tree`, `random_forest` (classical); `bilstm`, `gru`, `cnn`, `transformer` (deep). The autoencoder and the AlexNet+DenseNet+PCA+SVM hybrid are used programmatically (see `models/models.py`: `EEGAutoencoder / AutoencoderClassifierHead`, and `models/hybrid_alexnet_densenet.py`: `HybridAlexNetDenseNetPCASVM`) rather than via the CLI, since they need an extra unsupervised pretraining / image-construction step — see the docstrings in each file for a usage example.

Every run:

- fixes all random seeds (`utils.set_seed`, default 42),
- saves per-fold and mean/std accuracy + macro-F1 to `results/<model>_<task>_<cv>.json`.

6. Evaluation & visualization

```
from evaluation.evaluate import (
    plot_confusion_matrix, plot_roc_curve, plot_training_curves,
    plot_tsne, plot_feature_importance, plot_shap_summary, compare_with_paper,
)

plot_confusion_matrix(y_test, y_pred, class_names=["low", "high"])
plot_roc_curve(y_test, y_proba)
plot_tsne(X_features, y_test, class_names=["low", "high"])
plot_feature_importance(rf_model, feature_names=feature_names)
plot_shap_summary(rf_model, X_test, feature_names=feature_names,
model_type="tree")

compare_with_paper(
    reproduced={"random_forest_valence": 78.2, "bilstm_valence": 81.4},
    paper_reported={"random_forest_valence": 90.0, "bilstm_valence": 91.0},
)
```

All figures are written to `figures/`.

7. Comparing reproduced vs. paper-reported results

The paper reports a peak accuracy range of **85–95%** across its model sweep, and **95.54% (valence)** / **97.26% (arousal)** specifically for the AlexNet+DenseNet+PCA+SVM hybrid. When you run the pipeline end-to-end, record your numbers in a table like:

Model	Task	Paper-reported	Reproduced	CV protocol
Random Forest	valence	~85–90% (range)	(fill in)	subject-dependent
BiLSTM	valence	~85–90% (range)	(fill in)	subject-dependent
AlexNet+DenseNet+PCA+SVM	valence	95.54%	(fill in)	not specified
AlexNet+DenseNet+PCA+SVM	arousal	97.26%	(fill in)	not specified

If your reproduced numbers are lower, likely reasons (in rough order of impact)

1. **Subject-dependent vs. subject-independent (LOSO) evaluation.** Subject-dependent 10-fold CV on DEAP routinely scores 15–25 accuracy points higher than LOSO, because it lets the model see each subject's own EEG signature during training. If the paper's number is subject-dependent and you ran LOSO (or vice-versa), this alone explains most of the gap.
2. **Window length/overlap and label threshold**, which are assumed here (Section 1, items 3–4) rather than taken from the paper — different choices change the effective number of training samples and class balance.
3. **Hyperparameter tuning.** The classical models here use un-tuned literature defaults; a grid/random search over C/γ (SVM), depth/ $n_{\text{estimators}}$ (RF/DT), and k (KNN) typically adds several points.
4. **Hybrid model image construction.** The AlexNet+DenseNet+PCA+SVM pipeline's exact EEG → image transform is not specified in the paper; the per-channel-averaged spectrogram used here is one reasonable choice among several (per-channel stacked "multi-channel image", topographic scalp maps, etc.), and will materially change what the pretrained backbones "see."
5. **Feature set differences.** The paper names STFT and Wavelet Transform explicitly but the classical-ML feature vector composition (which exact statistics/bands) is not itemized; this repo concatenates PSD + DE + Hjorth + DWT + STFT band summaries, which may not exactly match what was fed to KNN/SVM/DT/RF in the original work.
6. **Random splits / seeds.** Fixed seeds make *this* pipeline reproducible run-to-run, but cannot reproduce the paper's own (unpublished) train/test split.

8. Notes on scientific coding standards

- Every stochastic component (NumPy, PyTorch, CUDA, sklearn splitters) is seeded via

`utils.set_seed.`

- Preprocessing, feature extraction, model definitions, training, and evaluation are separated into independent, independently-testable modules — no step is hidden inside a notebook.
- Every assumption not directly stated in the paper is called out explicitly (Section 1) rather than silently guessed.
- `dataset.py` fails loudly and informatively (`FileNotFoundError` with instructions) if the DEAP files are missing, rather than producing a misleading downstream error.